

Learning Flow Maps of Jump-Diffusion SDEs: A Runaway-Electron Target and a 1D Verification

June 17, 2026

Overview. The eventual target is a 2D runaway-electron (RE) model whose dynamics are naturally a stochastic differential equation (SDE) with jumps on a bounded momentum domain. We plan to learn its large-time stochastic flow map with a training-free diffusion model, plus an exit indicator for the boundary. Before the physics problem we verify the approach on a 1D bounded jump-diffusion, comparing the learned model against the classical numerical method in accuracy, speed, and long-horizon trajectory prediction.

1 The eventual target: a 2D runaway-electron model

The model. We track a single relativistic electron in 2D momentum space,

$$X_t = (p, \xi), \quad p = \text{normalized momentum}, \quad \xi = \cos \theta = \text{pitch-angle cosine}, \quad (1)$$

driven by the toroidal electric field, collisional drag and scattering, and rare large-angle collisions. The dynamics take the jump-diffusion form

$$dX_t = \underbrace{b(X_t) dt}_{\text{drift}} + \underbrace{\sigma(X_t) dW_t}_{\text{diffusion}} + \underbrace{\int \gamma(X_{t-}, z) \tilde{N}(dt, dz)}_{\text{jumps}}, \quad (2)$$

with the three pieces given concretely by

$$b_p = \hat{E} \xi - F_{\text{drag}}(p), \quad F_{\text{drag}}(p) \sim \frac{1+p^2}{p^2}, \quad (3)$$

$$b_\xi = \frac{\hat{E}(1-\xi^2)}{p}, \quad d\xi \supset \sqrt{2\nu_D(p)(1-\xi^2)} dW_\xi, \quad (4)$$

where $\hat{E} = E/E_c$ is the normalized field, F_{drag} is the collisional friction (it falls off as $\sim 1/v^2$ at high p , which is why runaways exist), and ν_D is the pitch-angle scattering rate. The momentum domain is bounded by the *runaway separatrix* p_c : below it the electron rejoins the thermal bulk (an exit/absorbing boundary).

Why it is an SDE with jumps. The full Coulomb collision operator splits into a small-angle part and a large-angle part. The small-angle (grazing) collisions accumulate into the Gaussian diffusion σdW (the classical Fokker–Planck picture). The rare large-angle, close (“knock-on”) collisions transfer a large momentum chunk in a single event; these are discrete, Poisson-timed, and heavy-tailed (Møller cross-section $d\Sigma/d\epsilon \propto \epsilon^{-2}$), i.e. exactly the jump term in Eq. (2) [4]. The same large-angle collisions drive the runaway *avalanche* [3]; here we keep the single-particle jump-diffusion and treat the avalanche as the motivating context. Framing runaway generation through the separatrix p_c as an exit problem matches existing RE methodology [5].

What we plan to do. Rather than integrating Eq. (2) with many small time steps, we will learn its *stochastic flow map* — the one-step transition density $p(X_{t+\Delta t} | X_t)$ at a *large* Δt — with a training-free diffusion model [1, 2], plus an exit indicator for the p_c boundary [2]. One evaluation of the learned map then advances the state by a large step, replacing many integrator sub-steps. The new ingredient relative to existing diffusion flow-map work is the *jump* term, which is the subject of the verification below.

2 A 1D verification example

We verify the approach on a 1D jump-diffusion in a *bounded* domain with an absorbing boundary — the structure the 2D runaway-electron problem needs (jumps, a large- Δt flow map, and exit across the separatrix). We learn two objects, a flow map for the in-domain transition and an *exit indicator* for the boundary, and compare against the classical numerical method in accuracy, speed, and long-horizon trajectory prediction.

The model. A 1D jump-diffusion on $[0, 6]$ with absorbing ends,

$$dX_t = \sigma_b dW_t + dJ_t, \quad X_t \in [0, 6], \quad (5)$$

with J_t a compound-Poisson process (rate 1, jumps $Y \sim \mathcal{N}(0, 0.7^2)$), $\sigma_b = 0.7$, and a large step $\Delta t = 1$. A particle that crosses either end leaves the domain (exits). The ground truth is a fine-step jump-adapted Euler–Maruyama integrator with the same boundaries.

The exit-indicator idea. Surviving particles stay inside $[0, 6]$ by construction, so the flow map is only ever evaluated in-domain — over arbitrarily long rollouts. We learn the boundary with a binary classifier $\mathcal{E}(x) = \Pr(\text{exit during } \Delta t | x)$, trained by cross-entropy on labeled survive/exit data (the JCP exit indicator of [2]). One coarse step is *exit, then propagate*: $\mathcal{E}$ removes the particles that cross the boundary, and the flow map — trained only on *surviving* transitions — advances the rest. Over a long rollout this tracks both the decaying surviving population and the surviving distribution, with the flow map always in-domain. This mirrors the runaway-electron separatrix p_c , where particles leave the runaway region into the thermal bulk.

What we did. Generate large- Δt data from the bounded SDE: survivor pairs $(X_t, X_{t+\Delta t})$ and exit labels $(X_t, \text{exit?})$. Learn the survivor flow map with the training-free method — learn the increment $X_{t+\Delta t} - X_t$ via a probability-flow ODE with a training-free Monte-Carlo score, distilled into a small network — and the exit indicator by cross-entropy. At inference, roll the two heads forward and compare against the numerical method.

Accuracy (one step). Figure 1 verifies the two heads in-domain: the exit indicator recovers the U-shaped exit probability (high at the walls, near zero mid-domain), and the survivor transition density matches the numerical truth ($W_1 = 0.004$ and 0.015 at $x = 1.5, 3.0$).

Long-horizon trajectory and speed. Figure 2 propagates to $T = 8$ ($K = 8$ big ML steps vs. the integrator’s $8 \times 200 = 1600$ sub-steps). The ML rollout tracks the *surviving fraction* at every step (final 0.316 vs. 0.318, a 0.6% error) and reproduces the *terminal surviving density* ($W_1 = 0.010$, near the sampling floor), about $40\times$ faster than the numerical method. The structural reason for the speedup is that the learned map advances the state in one big step rather than many sub-steps.

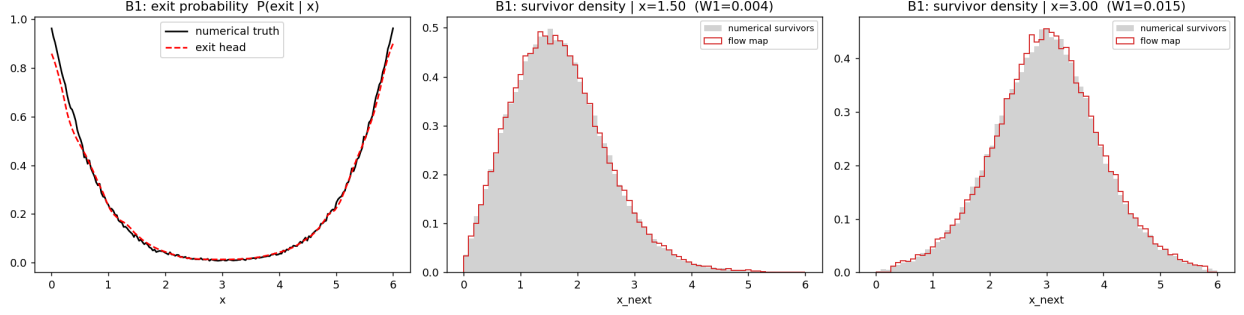


Figure 1: One step. Left: exit probability $\Pr(\text{exit} | x)$, exit indicator (red) vs. numerical (black). Middle/right: survivor transition density, flow map (red) vs. numerical survivors (gray), at $x = 1.5, 3.0$.

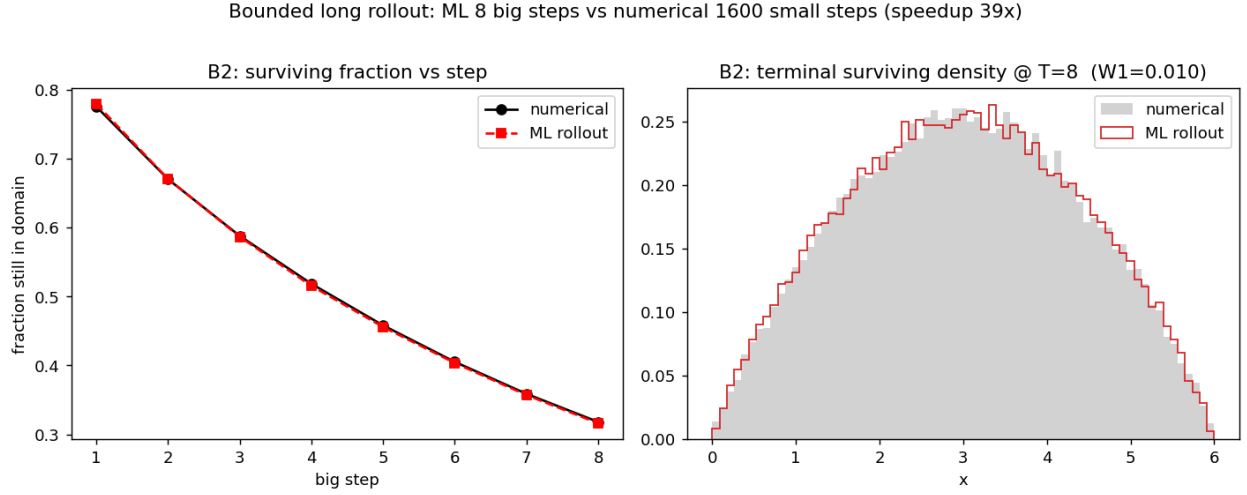


Figure 2: Long rollout to $T = 8$. Left: surviving fraction vs. step, ML rollout (red) vs. numerical (black). Right: terminal surviving density at $T = 8$, ML (red) vs. numerical (gray), $W_1 = 0.010$. The ML map takes 8 big steps vs. the integrator’s 1600 small steps. (Wall-clock is indicative: integrator on CPU, network on GPU; the hardware-independent statement is 8 vs. 1600 steps.)

Conclusion. On a 1D bounded jump-diffusion, the learned flow map plus exit indicator reproduce both the survival dynamics and the terminal distribution over a long horizon, at a large Δt and about $40\times$ faster than the classical integrator. This exercises every ingredient the 2D runaway-electron model of Eq. (2) requires — jumps, a large- Δt flow map, and an exit indicator for the separatrix — before scaling up.

References

- [1] Y. Liu, M. Yang, Z. Zhang, F. Bao, Y. Cao, G. Zhang, “Diffusion-model-assisted supervised learning of generative models for density estimation,” *J. Machine Learning for Modeling and Computing* **5**(1), 25–38 (2024). doi:10.1615/JMachLearnModelComput.2024051346.
- [2] M. Yang, Y. Liu, D. del-Castillo-Negrete, Y. Cao, G. Zhang, “Generative AI models for learning

- flow maps of stochastic dynamical systems in bounded domains,” *J. Comput. Phys.*, 114434 (2025).
- [3] M. N. Rosenbluth, S. V. Putvinski, “Theory for avalanche of runaway electrons in tokamaks,” *Nucl. Fusion* **37**, 1355 (1997).
- [4] O. Embréus, A. Stahl, T. Fülöp, “On the relativistic large-angle electron collision operator for runaway avalanches in plasmas,” *J. Plasma Phys.* **84**, 905840102 (2018).
- [5] M. Yang, G. Zhang, D. del-Castillo-Negrete, M. Stoyanov, “A Feynman–Kac based numerical method for the exit time probability of a class of transport problems,” *J. Comput. Phys.* **444**, 110564 (2021). doi:10.1016/j.jcp.2021.110564.